SIMATS ENGINEERING

ASSESSMENT TOOL 2-Pseudocode Writing Task

COURSE NAME: R PROGRAMMING **COURSE CODE: ITA04**

COURSE OUTCOME COVERED

CO1: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively. Identify and implement appropriate control structures to solve a particular programming problem
(Bloom Level -BL3- BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Questions **Total Marks: 100**

Code of Conduct:

I vignesh(192525013*)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *vignesh*

Q1. Campus Canteen Billing

Concepts: Vectors, element-wise multiplication, sum, IF–ELSE

Pseudocode:

```
BEGIN
INPUT price[1..n], qty[1..n]
FOR i ← 1 TO n
   itemBill[i] ← price[i] × qty[i]
END FOR
total ← SUM(itemBill)
IF total > 500 THEN
   discount ← total × 0.10
ELSE
   discount ← 0
END IF
payable ← total − discount
DISPLAY itemBill, total, discount, payable
END
```

Q2. Student Marks Analyser

Concepts: Vectors, counting, minimum, maximum, average, IF–ELSE

Pseudocode:

```
BEGIN
INPUT marks[1..40]
average ← SUM(marks) / 40
highest ← marks[1]; lowest ← marks[1]
failCount ← 0
FOR i ← 1 TO 40
   IF marks[i] > highest THEN highest ← marks[i]
   IF marks[i] < lowest THEN lowest ← marks[i]
   IF marks[i] < 40 THEN failCount ← failCount + 1
END FOR
passCount ← 40 − failCount
passPercent ← (passCount / 40) × 100
IF passPercent > 80 THEN
   DISPLAY "Class Performance: Good"
ELSE
   DISPLAY "Needs Improvement"
END IF
DISPLAY average, highest, lowest, failCount, passCount, passPercent
END
```

Q3. Hostel Room Allocation Matrix

Concepts: 4 × 5 matrix, nested loops, row-wise counting, maximum

Pseudocode:

```
BEGIN
```

```
INPUT room[1..4, 1..5]   // 1 = occupied, 0 = vacant
FOR floor ← 1 TO 4
   vacant[floor] ← 0
   FOR roomNo ← 1 TO 5
      IF room[floor, roomNo] = 0 THEN
         vacant[floor] ← vacant[floor] + 1
      END IF
   END FOR
   DISPLAY "Floor", floor, "vacant rooms:", vacant[floor]
END FOR
totalVacant ← SUM(vacant[1..4])
maxVacant ← vacant[1]; maxFloor ← 1
FOR floor ← 2 TO 4
   IF vacant[floor] > maxVacant THEN
      maxVacant ← vacant[floor]; maxFloor ← floor
   END IF
END FOR
DISPLAY totalVacant, maxFloor, maxVacant
END
```

Q4. Monthly Electricity Bill Calculator

Concepts: IF–ELSE ladder, slab-based calculation

Pseudocode:

```
BEGIN
INPUT units
IF units < 0 THEN
   DISPLAY "Invalid units"; STOP
END IF
IF units <= 100 THEN
   energyCharge ← units × 3
ELSE IF units <= 200 THEN
   energyCharge ← (100 × 3) + ((units − 100) × 5)
ELSE
   energyCharge ← (100 × 3) + (100 × 5) + ((units − 200) × 8)
END IF
fixedCharge ← 50
totalBill ← energyCharge + fixedCharge
DISPLAY units, energyCharge, fixedCharge, totalBill
END
```

Q5. Sales Report Using Matrix Operations

Concepts: 3 × 4 matrix, row and column sums, maximum, transpose

Pseudocode:

```
BEGIN
INPUT sales[1..3, 1..4]   // rows = products, columns = weeks
FOR product ← 1 TO 3
```

```
    productTotal[product] ← 0
    FOR week ← 1 TO 4
      productTotal[product] ← productTotal[product] + sales[product, week]
    END FOR
  END FOR
  FOR week ← 1 TO 4
    weekTotal[week] ← 0
    FOR product ← 1 TO 3
      weekTotal[week] ← weekTotal[week] + sales[product, week]
    END FOR
  END FOR
  bestProduct ← index of maximum value in productTotal
  bestWeek ← index of maximum value in weekTotal
  transposed[1..4, 1..3] ← TRANSPOSE(sales)
  DISPLAY productTotal, weekTotal, bestProduct, bestWeek, transposed
  END
```

Q6. Library Book Management Using Lists

Concepts: List of lists, traversal, condition checking, update

Pseudocode:

```
  BEGIN
  CREATE books[1..5]; each book has title, author, year, availability
  FOR i ← 1 TO 5
    INPUT books[i].title, books[i].author, books[i].year, books[i].availability
  END FOR
  FOR i ← 1 TO 5
    IF books[i].year > 2015 THEN DISPLAY books[i]
  END FOR
  INPUT borrowTitle
  found ← FALSE
  FOR i ← 1 TO 5
    IF books[i].title = borrowTitle THEN
      found ← TRUE
      IF books[i].availability = TRUE THEN
        books[i].availability ← FALSE
        DISPLAY "Book borrowed successfully"
      ELSE
        DISPLAY "Book is already unavailable"
      END IF
      BREAK
    END IF
  END FOR
  IF found = FALSE THEN DISPLAY "Title not found"
  END
```

Q7. Temperature Data Cleaning

Concepts: NA handling, mean, loops, classification vector

Pseudocode:

```
BEGIN
INPUT temperature[1..30]
missingCount ← 0; validSum ← 0; validCount ← 0
FOR i ← 1 TO 30
   IF temperature[i] is NA THEN
      missingCount ← missingCount + 1
   ELSE
      validSum ← validSum + temperature[i]
      validCount ← validCount + 1
   END IF
END FOR
IF validCount = 0 THEN DISPLAY "No valid readings"; STOP
meanTemp ← validSum / validCount
FOR i ← 1 TO 30
   IF temperature[i] is NA THEN temperature[i] ← meanTemp
END FOR
FOR i ← 1 TO 30
   IF temperature[i] < 20 THEN category[i] ← "Cold"
   ELSE IF temperature[i] <= 30 THEN category[i] ← "Moderate"
   ELSE category[i] ← "Hot"
END FOR
DISPLAY missingCount, meanTemp, temperature, category
END
```

Q8. Exam Seating Using 3D Arrays

Concepts: 3D array, nested loops, occupied-seat counting

Pseudocode:

```
BEGIN
INPUT seat[1..3, 1..4, 1..5]   // 0 means empty
FOR block ← 1 TO 3
   occupied[block] ← 0
   FOR row ← 1 TO 4
      FOR column ← 1 TO 5
         IF seat[block, row, column] ≠ 0 THEN
            occupied[block] ← occupied[block] + 1
         END IF
      END FOR
   END FOR
   DISPLAY "Block", block, "occupied seats:", occupied[block]
END FOR
bestBlock ← index of maximum value in occupied
```

```
registerNo ← seat[2, 3, 4]
DISPLAY bestBlock, occupied[bestBlock], registerNo
END
```

Q9. Employee Data Frame Processing

Concepts: Data frame filtering, grouping, conditional update, new column

Pseudocode:

```
BEGIN
INPUT employee data frame with EmpID, Name, Department, Salary
FOR each employee e
   IF e.Department = "IT" AND e.Salary > 50000 THEN DISPLAY e
END FOR
FOR each distinct department d
   deptSum ← 0; deptCount ← 0
   FOR each employee e
      IF e.Department = d THEN
         deptSum ← deptSum + e.Salary
         deptCount ← deptCount + 1
      END IF
   END FOR
   IF deptCount > 0 THEN deptAverage[d] ← deptSum / deptCount
END FOR
FOR each employee e
   IF e.Salary < 30000 THEN NewSalary[e] ← e.Salary × 1.10
   ELSE NewSalary[e] ← e.Salary
END FOR
ADD NewSalary as a new column to employee data frame
DISPLAY filtered IT employees, deptAverage, updated data frame
END
```

Q10. Prime Number and Fibonacci Series Generator

Concepts: Input validation, nested loops, vectors, prime checking

Pseudocode:

```
BEGIN
INPUT N
WHILE N is not a positive integer
   DISPLAY "Enter a positive integer"
   INPUT N
END WHILE
primes ← empty vector
FOR number ← 2 TO N
   isPrime ← TRUE
   FOR divisor ← 2 TO FLOOR(SQRT(number))
      IF number MOD divisor = 0 THEN isPrime ← FALSE; BREAK
   END FOR
   IF isPrime = TRUE THEN APPEND number TO primes
```

```
END FOR
primeCount ← LENGTH(primes)
primeSum ← SUM(primes)
DISPLAY primes, primeCount, primeSum
fibonacci ← empty vector
IF N >= 1 THEN APPEND 0 TO fibonacci
IF N >= 2 THEN APPEND 1 TO fibonacci
WHILE LENGTH(fibonacci) < N
   nextTerm ← last(fibonacci) + secondLast(fibonacci)
   APPEND nextTerm TO fibonacci
END WHILE
DISPLAY first N terms of fibonacci
END
```

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/ modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient